DATA STRUCTURES

CO2 DEBUGGING

Assessment Questions & Corrected Answers

Name	PAPAGARI PUSHPA CHARITHA
Reg No	192525371
Course	DATA STRUCTURES
Course Code	CSA0312

CO2 • DEBUGGING
3 Assessment Questions • 5 Marks Each

QUESTION 1

6. Buggy Code in underflow operation (5 marks)

Question: Find and fix the bug in the stack pop() operation.

Given buggy code:

```
int pop() {
    if(top == 0) {
        printf("Underflow");
        return -1;
    }
    return stack[top--];
}
```

Answer / Bug Identified:

The stack is empty when top == -1, not when top == 0. Using top == 0 incorrectly reports underflow while one element is still present. The corrected condition is if(top == -1).

Corrected Program:

```
#include <stdio.h>

#define MAX 100

int stack[MAX];
int top = -1;

int pop() {
    if(top == -1) {
        printf("Underflow\n");
        return -1;
    }
    return stack[top--];
}

void push(int value) {
    if(top == MAX - 1) {
        printf("Overflow\n");
        return;
    }
    stack[++top] = value;
}

int main() {
    push(10);
    push(20);
    push(30);

    printf("Popped: %d\n", pop());
    printf("Popped: %d\n", pop());
    printf("Popped: %d\n", pop());
    printf("Popped: %d\n", pop());

    return 0;
}
```

Expected output:

```
Popped: 30
Popped: 20
Popped: 10
Underflow
Popped: -1
```

QUESTION 2

9. Include missing statements and display the list of elements in linkedlist (5 marks)

Question: Correct the linked-list display function and include the required statements so the list elements are displayed.

Given buggy code:

```
void display(struct Node *head)
{
    while(head != NULL);
    {
        printf("%d ", head->data);
        head = head->next;
    }
}
```

Answer / Bug Identified:

The error is the semicolon after while(head != NULL);. That semicolon makes the loop body empty. The semicolon must be removed so the printf statement and the pointer update execute inside the loop.

Corrected Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

void display(struct Node *head)
{
    while(head != NULL)
    {
        printf("%d ", head->data);
        head = head->next;
    }
}

int main()
{
    struct Node *first = malloc(sizeof(struct Node));
    struct Node *second = malloc(sizeof(struct Node));
    struct Node *third = malloc(sizeof(struct Node));

    first->data = 10;
    first->next = second;

    second->data = 20;
    second->next = third;

    third->data = 30;
    third->next = NULL;

    display(first);

    free(first);
    free(second);
    free(third);

    return 0;
}
```

Expected output:

```
10 20 30
```

QUESTION 3

Identify an errors in the following snippet (5 marks)

Question: Identify and fix the errors in the binary search implementation.

Given buggy code:

```
int binarySearch(int arr[], int n, int key)
{
    int low = 0, high = n - 1;

    while(low < high)
    {
        int mid = (low + high) / 2;

        if(arr[mid] == key)
            return mid;
        else if(arr[mid] < key)
            low = mid;
        else
            high = mid;
    }

    return -1;
}
```

Answer / Bugs Identified:

There are three logic errors:
1. The loop condition should be low <= high so the final remaining element is checked.
2. When arr[mid] < key, update low = mid + 1, not low = mid.
3. When arr[mid] > key, update high = mid - 1, not high = mid.

Corrected Program:

```
#include <stdio.h>

int binarySearch(int arr[], int n, int key)
{
    int low = 0, high = n - 1;

    while(low <= high)
    {
        int mid = (low + high) / 2;

        if(arr[mid] == key)
            return mid;
        else if(arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }

    return -1;
}

int main()
{
    int arr[] = {2, 4, 6, 8, 10, 12, 14};
    int n = sizeof(arr) / sizeof(arr[0]);
    int key = 10;

    int result = binarySearch(arr, n, key);

    if(result != -1)
        printf("Element found at index %d\n", result);
    else
        printf("Element not found\n");

    return 0;
}
```

Expected output:

```
Element found at index 4
```

QUICK REVISION — FINAL ANSWERS

Q1	Stack underflow: use top == -1 for an empty stack.
Q2	Linked list: remove the extra semicolon after the while condition.
Q3	Binary search: use low <= high, mid + 1, and mid - 1.

Assessment Summary

This document arranges all three CO2 Debugging assessment questions in Question → Bug → Corrected Answer → Expected Output order for easy submission and revision.

Student Details

Name	PAPAGARI PUSHPA CHARITHA
Reg No	192525371
Course	DATA STRUCTURES
Course Code	CSA0312